

AWS CI/CD Pipeline for Order Cloud Lambda

1. Aim

To create and verify a Continuous Integration and Continuous Deployment (CI/CD) workflow for an AWS Lambda-based order-processing function using GitHub as the source repository, AWS CodePipeline for automated deployment, Amazon S3 for source artifacts, Amazon CloudWatch for execution logs, and Amazon EventBridge with Amazon SNS for success notifications.

2. Tools and Services Used

GitHub — source-code repository.

Amazon S3 — storage location for pipeline source artifacts.

AWS CodePipeline — source and deployment stages.

AWS Lambda — serverless OrderCloudOrderProcessor function.

Amazon CloudWatch Logs — runtime execution evidence.

Amazon EventBridge — rule matching successful pipeline state changes.

Amazon SNS and Gmail — delivery of pipeline-success notifications.

3. Workflow Overview

GitHub source → Code Pipeline → Lambda deployment

↓ CloudWatch logs • Event Bridge rule → SNS email notification

The screenshots show a pipeline named Order Cloud-Lambda-CICD. Its Source stage uses GitHub, and its Deploy stage deploys to AWS Lambda. The pipeline execution shown in the evidence completed successfully.

4. Procedure

4.1 Create and maintain the source repository

Store the Lambda handler in the GitHub repository. The supplied repository screenshot shows `lambda_function.py` and `README.md`. The handler logs a deployment-verification message, reads `order_id` from the event (using `ORD-1001` as a fallback), logs the order being processed, and returns an HTTP-style `statusCode` of 200 with a success message and order ID.

4.2 Configure the CI/CD pipeline

Create the OrderCloud-Lambda-CICD pipeline with a GitHub source stage and an AWS Lambda deploy stage. The provided pipeline screenshot shows green success indicators for both stages.

4.3 Store and inspect source artifacts

Open the `SourceArt/` prefix in the pipeline's S3 bucket and confirm that source-artifact objects are present. The supplied screenshot shows three objects in this prefix.

4.4 Deploy and test the Lambda function

Open the OrderCloudOrderProcessor function in AWS Lambda. Run the saved test event named OrderTest. The supplied test output shows Status: Succeeded, `statusCode` 200, and the message "Order processed successfully."

4.5 Inspect runtime logs

Open the function's CloudWatch log stream. Verify that the log entries include the Lambda start, the message that the order processor started, processing of order `ORD-1001`, and the end/report records.

4.6 Verify Lambda versions

Inspect the Lambda Versions tab. The screenshot shows versions 1 through 4; version 1 has the `PRODUCTION` alias. This demonstrates that published versions are present, but it does not by itself prove that every version was invoked successfully.

4.7 Configure success-event notification

Use an EventBridge rule named OrderCloud-Pipeline-Success. Its event pattern matches source `aws.codepipeline`, detail-type `CodePipeline Pipeline Execution State Change`, state `SUCCEEDED`, and pipeline `OrderCloud-Lambda-CICD`. The rule is shown as Enabled.

4.8 Confirm email notification

Check the notification email. The provided AWS Notification Message reports the pipeline execution state as `SUCCEEDED` for OrderCloud-Lambda-CICD.

5. Screenshots and Observations

The following selected screenshots document the source repository, deployment, test response, artifact storage, runtime logs, Lambda versions, event rule, and notification. Repeated screenshots of the same Lambda Versions and CloudWatch pages were omitted to keep the report concise.

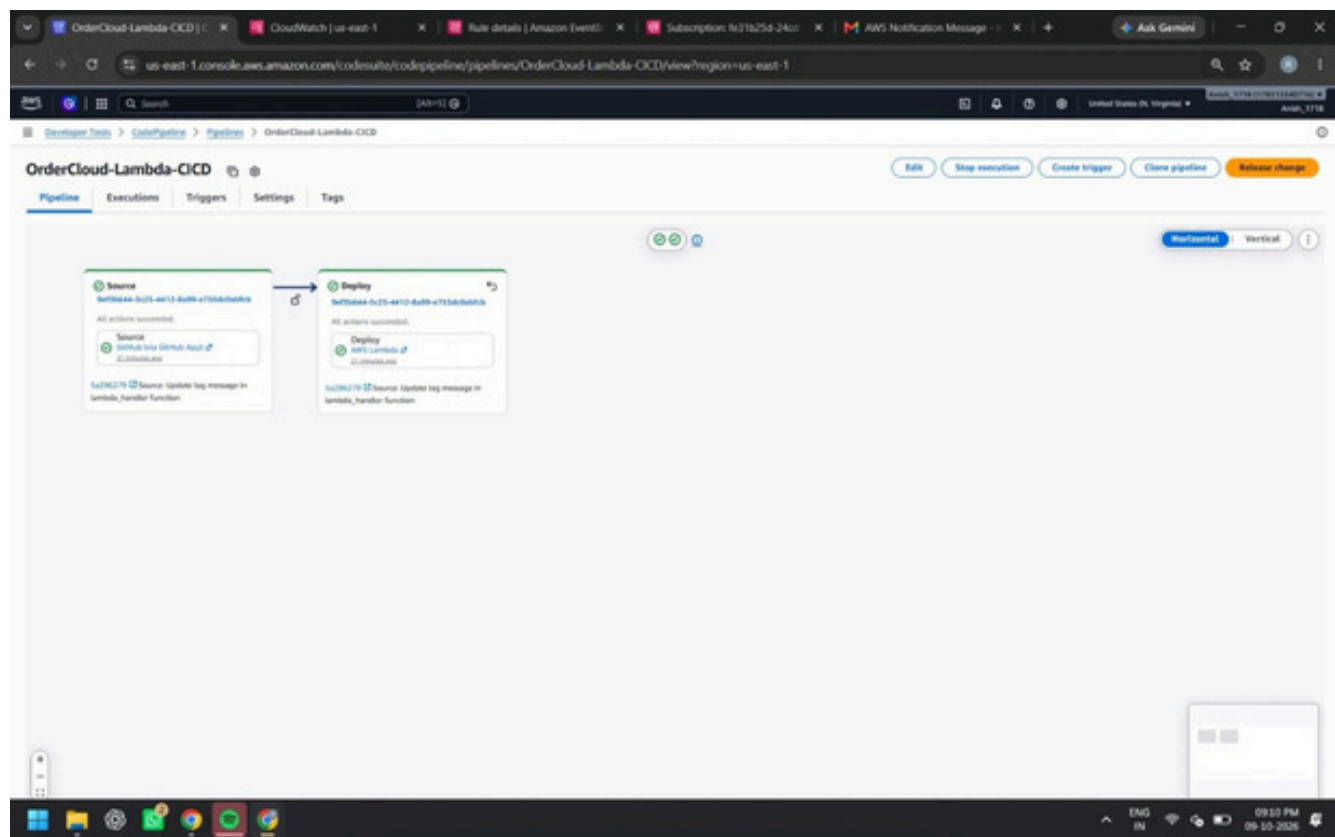


Figure 1. AWS CodePipeline showing successful Source and Deploy stages.

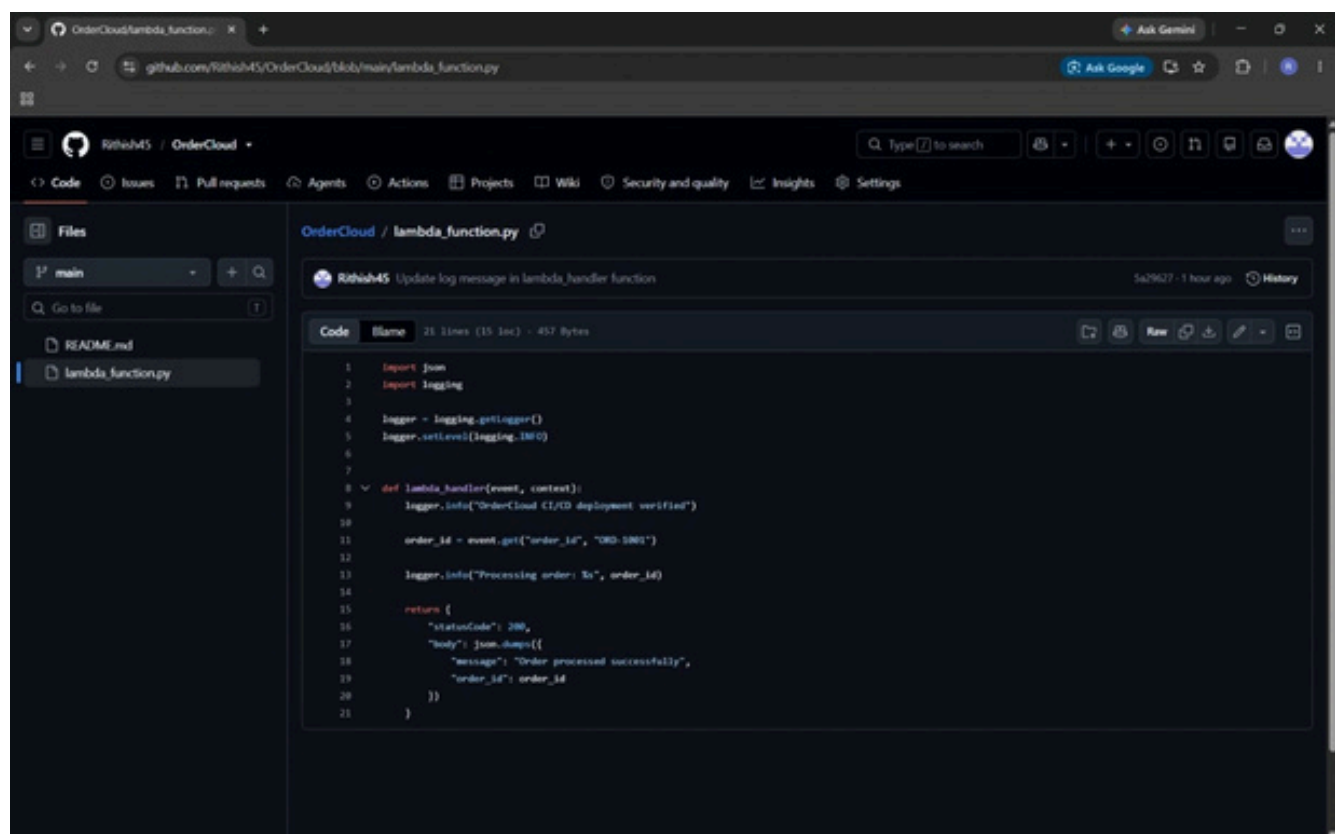


Figure 2. GitHub repository containing the Lambda function source code.

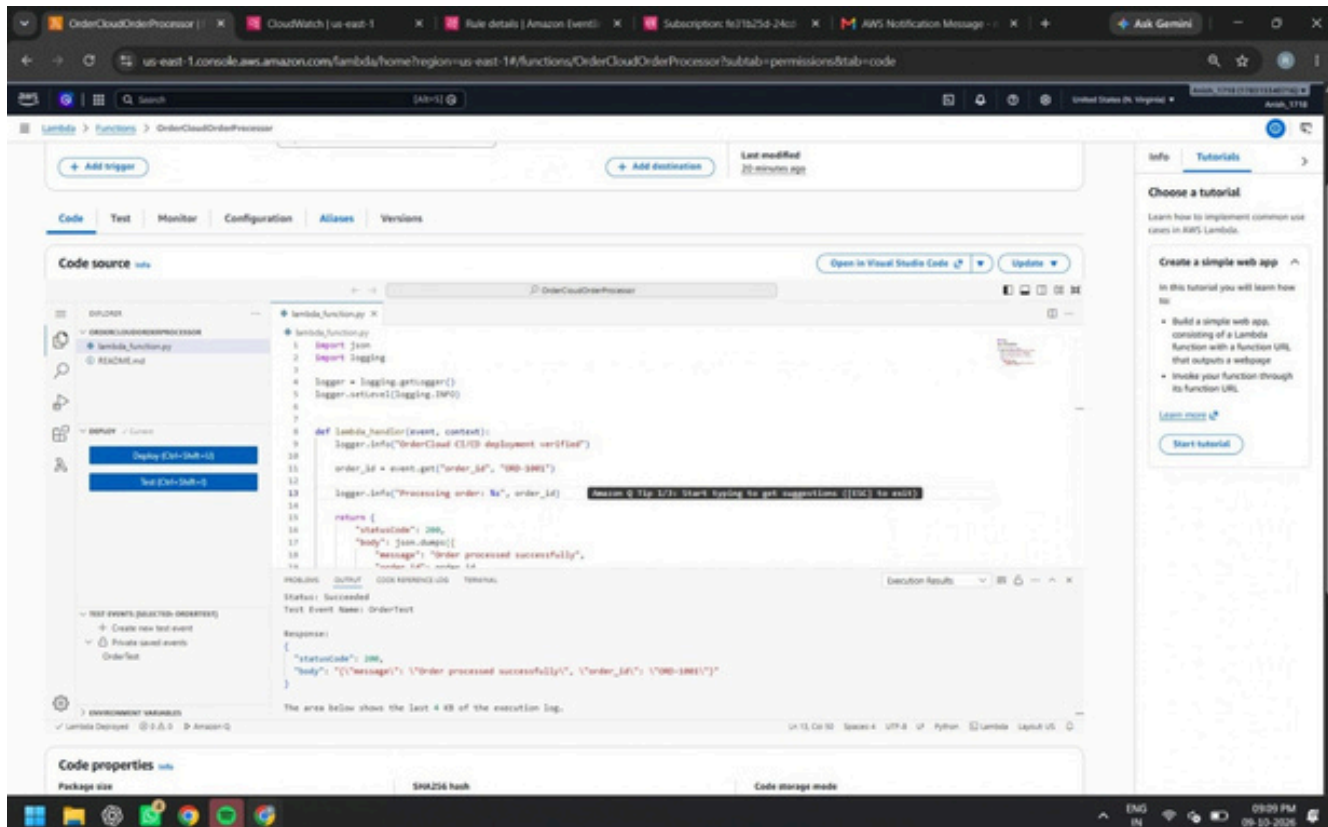


Figure 3. AWS Lambda code editor and successful test execution returning statusCode 200.

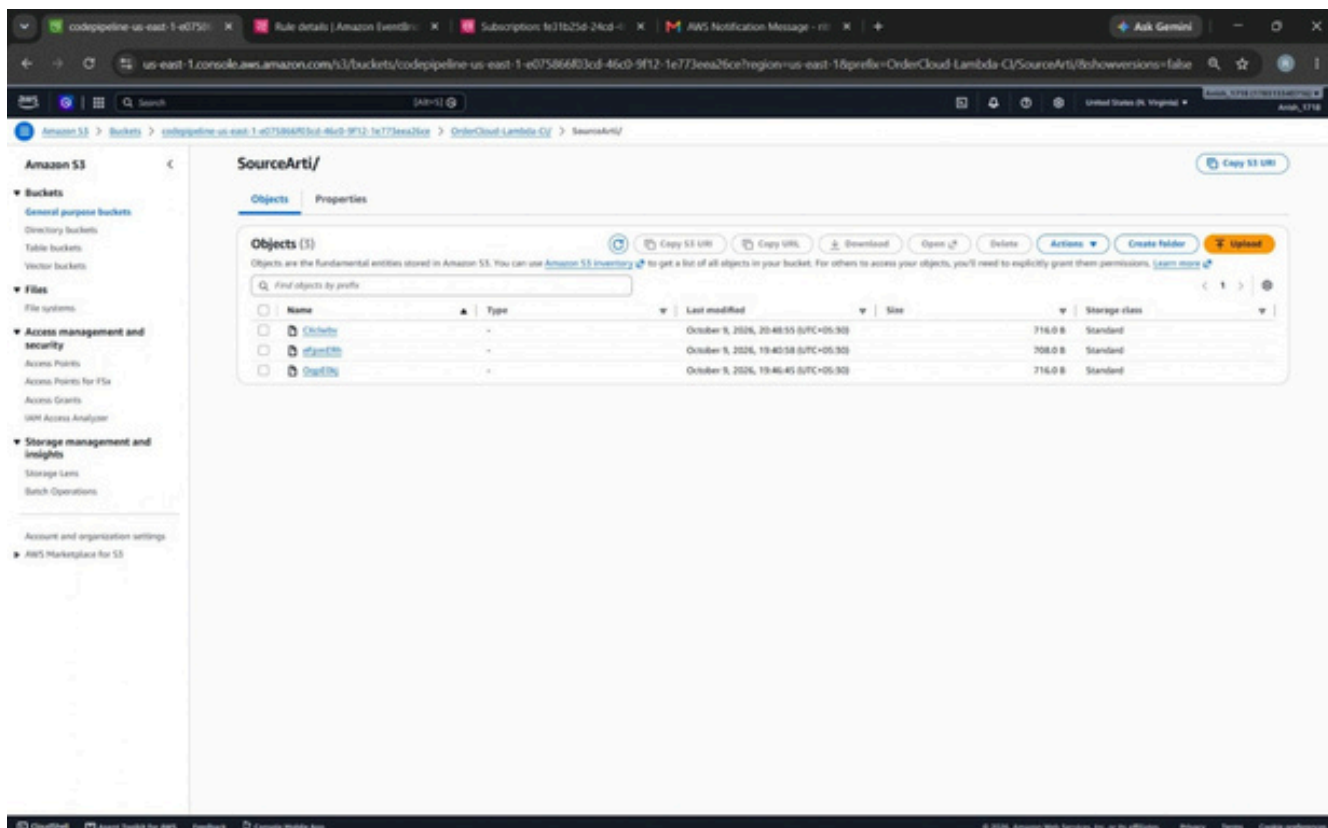


Figure 4. Amazon S3 SourceArt/ prefix containing the pipeline source artifacts.

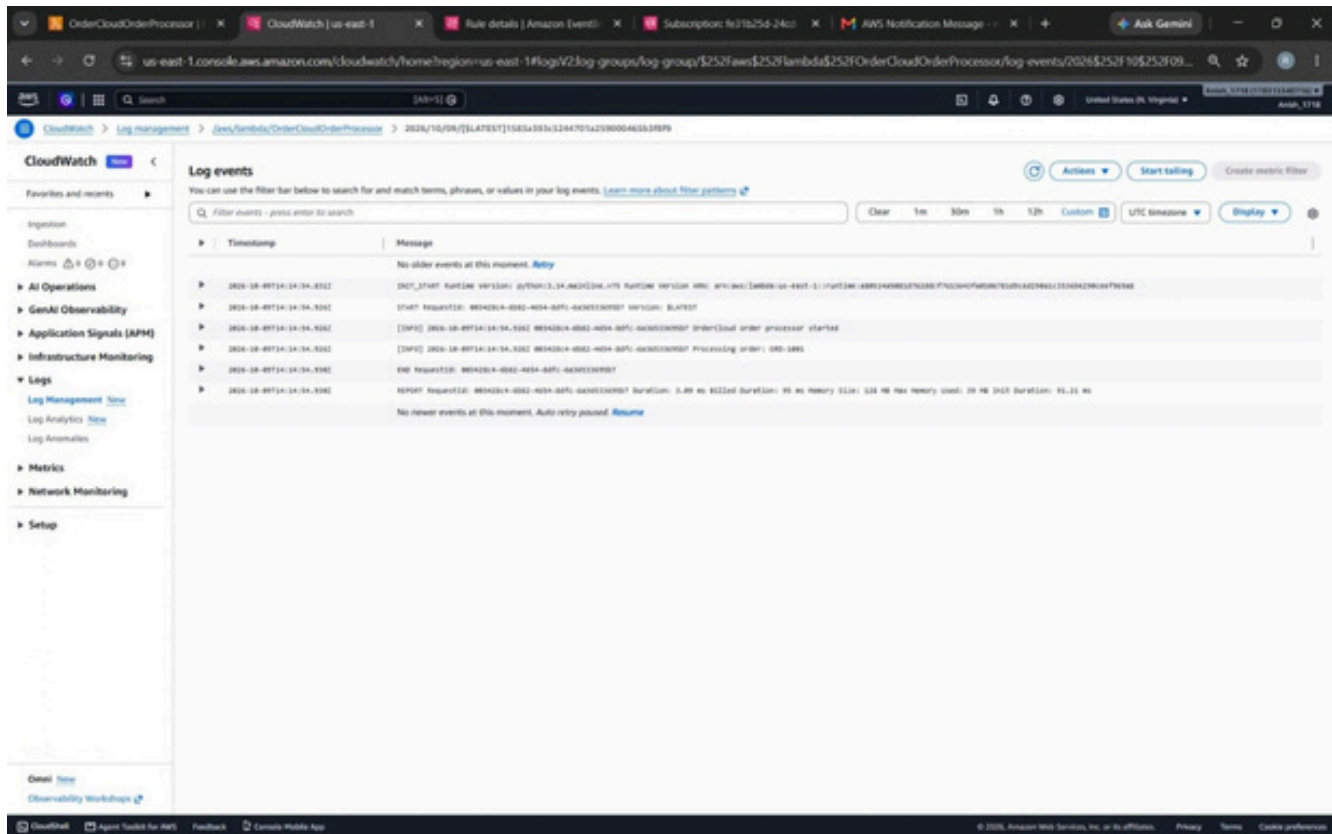


Figure 5. Amazon CloudWatch logs showing the order processor start, order processing, and successful completion.

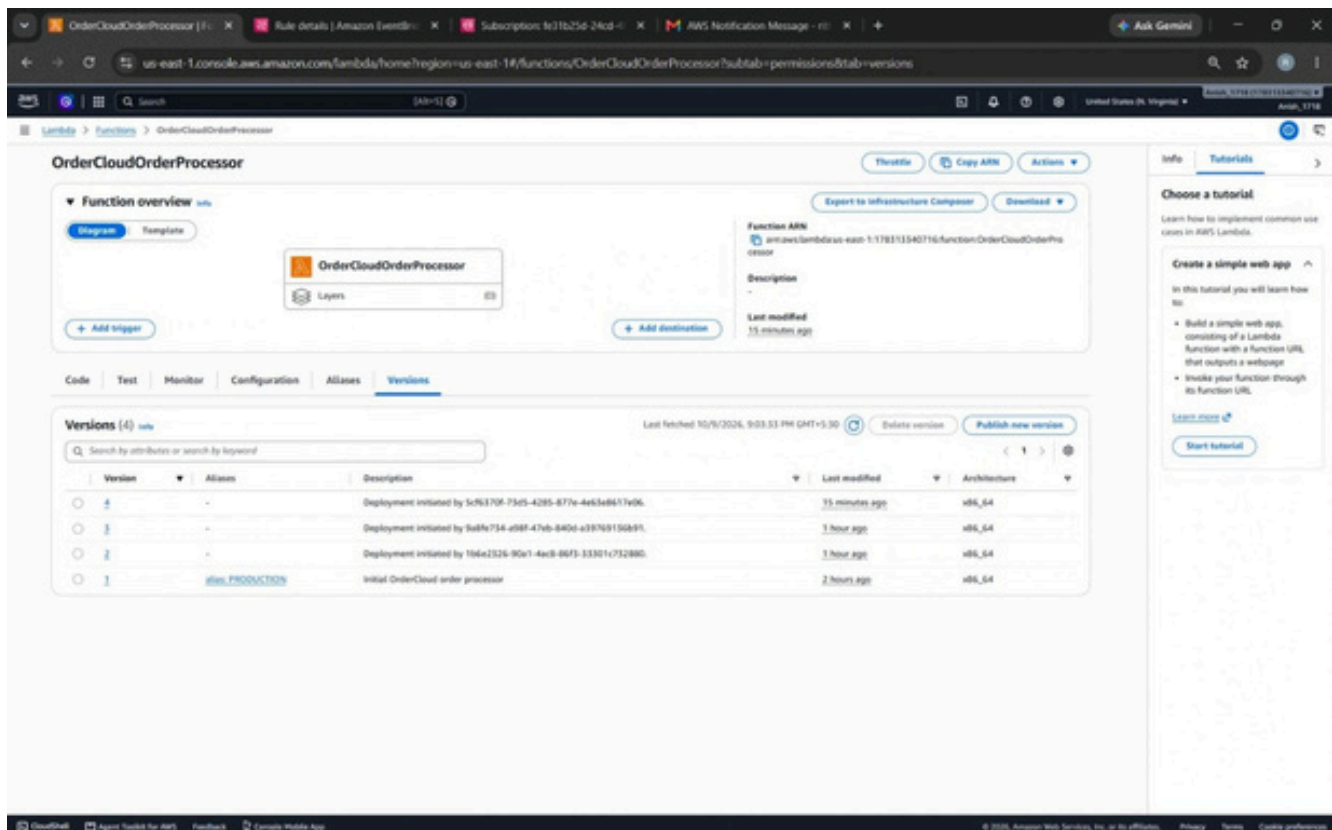


Figure 6. AWS Lambda versions page showing published versions and the PRODUCTION alias on version 1.

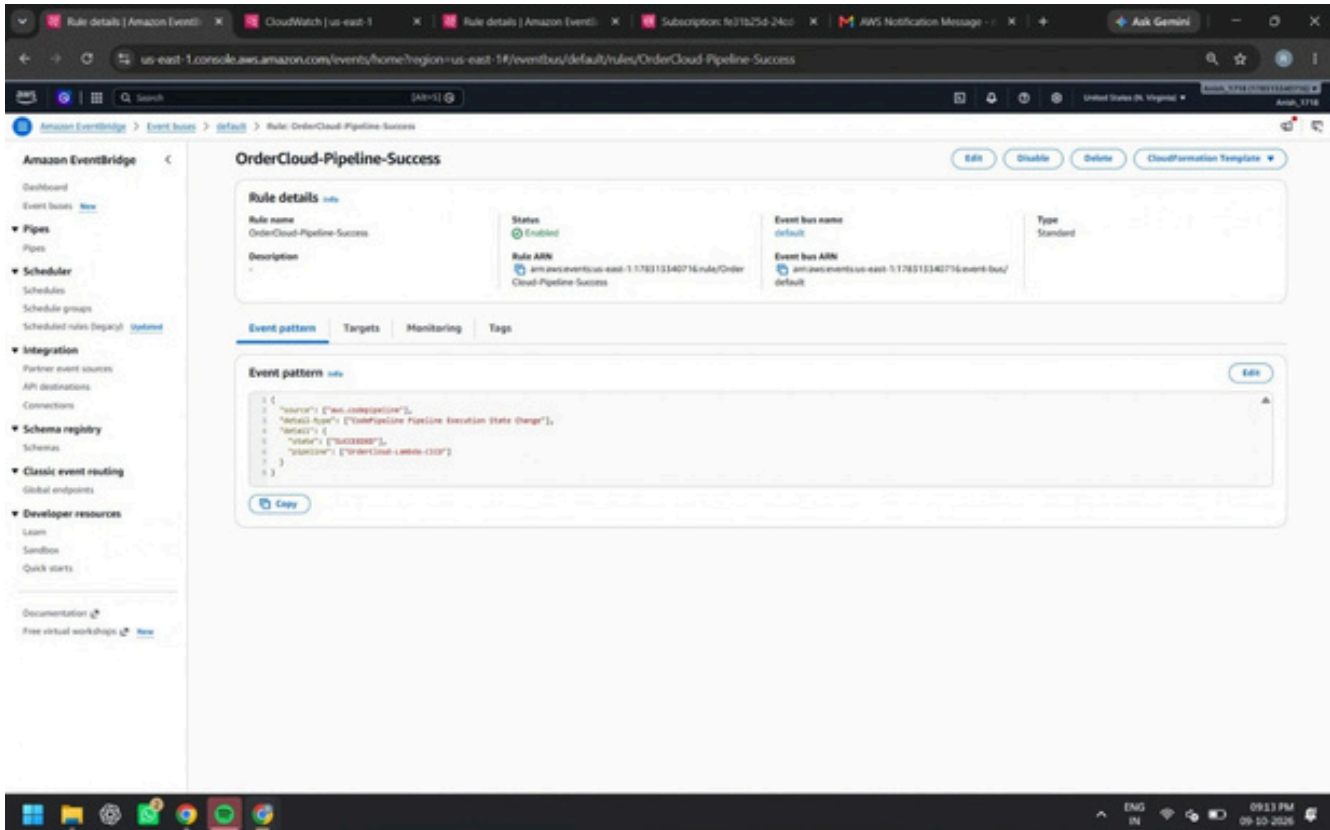


Figure 7. Amazon EventBridge rule configured for a successful CodePipeline execution.

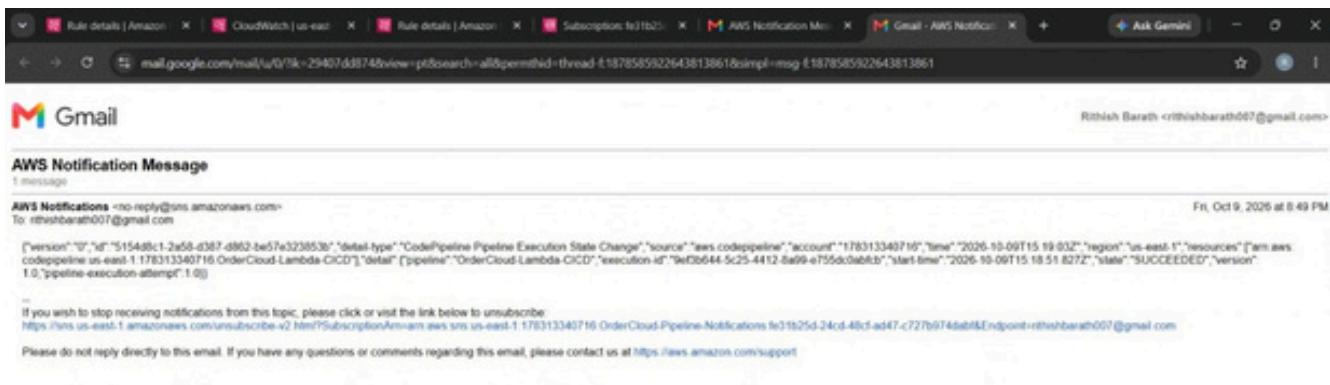


Figure 8. AWS SNS notification email reporting the pipeline execution state as SUCCEEDED.

6. Result

The AWS CI/CD workflow for the OrderCloud Lambda function was demonstrated successfully in the supplied evidence. The CodePipeline Source and Deploy stages completed successfully, the Lambda test returned statusCode 200, CloudWatch recorded the order-processing messages, the EventBridge success rule was enabled, and an AWS SNS notification email reported the pipeline execution state as SUCCEEDED.